

RISANSI INDUSTRIES LTD

Pump Testing Portal

Technical Documentation

rndtesting.risansi.com

Version: working tree as of 5 October 2026

Audience: Developers and maintainers

Status: Internal document

1. Overview

testing-portal is a Next.js 15 (App Router) application written in TypeScript. The browser app and the backend live in one repository: pages render in React, and the API is a set of Next.js route handlers under `src/app/api`. Data is stored in an Azure PostgreSQL database (database name `Pump_Selector_and_testing`) accessed through Drizzle ORM and `node-postgres`.

Item	Value
Framework	Next.js 15.5 (App Router), React 19, TypeScript
Database access	Drizzle ORM 0.45 with pg 8 (node-postgres) against Azure PostgreSQL
Forms and validation	react-hook-form 7 with zod 4 (@hookform/resolvers)
Authentication	Hand-written JWT (HS256) with jsonwebtoken; passwords hashed with bcryptjs
HTTP client	axios, wrapped in <code>src/services/apiClient.ts</code>
Styling	Tailwind CSS v4 utilities mapped to CSS design tokens, plus per-screen CSS files
PDF	Server: pdfkit 0.15.2 (pinned) for the audit report. Browser: <code>window.print()</code> with a print stylesheet for test reports
Hosting	Vercel, deployed on every push to main of <code>github.com/farhan2312/risansa-testing</code>
Live URL	<code>https://rndtesting.risansa.com</code>
Sister app	<code>sales-portal-next</code> (separate repo and deployment) sharing the users table and JWT secret

2. Architecture

2.1 Request flow

- A page file in `src/app/(dashboard)/...` is a thin wrapper that re-exports a screen component from `src/screens/...`
- The screen calls a function in `src/services/*` (axios) which calls an API route under `/api`.
- The route handler authenticates the caller from the `auth_token` cookie, checks the role, queries the database through Drizzle, and returns `snake_case` JSON built by the serializers in `src/lib/api.ts`.
- Shared business logic (calculations, permissions, categories) lives in `src/lib` so the UI and the API apply the same rule.

2.2 Repository layout

Path	Contents
<code>src/app/(dashboard)/</code>	Thin <code>page.tsx</code> wrappers for every signed-in screen
<code>src/app/api/</code>	Route handlers: the backend
<code>src/screens/</code>	The real page components (business logic and JSX), one folder per screen
<code>src/components/</code>	ui (modals, headers, pagination), charts (cards, bars, donut, heat map), test-report forms, report-detail, auth guard
<code>src/layouts/DashboardLayout.tsx</code>	Sidebar, top bar, profile menu, forced-password dialog, notification bell
<code>src/lib/</code>	Auth, DB, calculations, permissions, serializers, date and audit helpers

private-docs/	The documentation PDFs, kept out of /public and served only by /api/admin-docs/[name] to admins
src/lib/db/	schema.ts (Drizzle tables) and a lazy connection pool
src/services/	Client-side fetch wrappers
src/contexts/	AuthContext (user held in memory) and ThemeContext
src/types/testing.ts	Shared TypeScript types and constants (categories, statuses, persons, units)
scripts/	One-off .mjs migration and data scripts

3. Environment, build and run

Configuration is read from .env.local (never committed). Required keys:

Variable	Purpose
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	PostgreSQL connection
DB_SSLMODE	require for Azure, disable for local
JWT_SECRET	Signing key. Must be identical to sales-portal-next's JWT_SECRET so a login works in both apps

The database pool is created lazily through a Proxy, so next build does not fail when credentials are absent at build time.

Command	What it does
npm run dev	Dev server on http://localhost:5174
npm run build	Production build (includes lint and type-check)
npm run start	Serve the build on port 5174
npx tsc --noEmit	Type-check only. Run after every change
npx dotenv -e .env.local -- node scripts/xxx.mjs	Run a one-off database script
npm run db:generate / db:migrate / db:push	drizzle-kit commands

4. Authentication and security

- Login (POST /api/auth/login) checks the bcrypt hash and issues a JWT with sub (user id), email, role, iat and exp. Expiry is 12 hours.
- The token is stored in a cookie named auth_token that is httpOnly, Secure and SameSite=Lax. Browser code never reads it. GET /api/auth/me verifies it and AuthContext keeps the result in memory only.
- decodeToken(req) verifies any valid token. requireAdmin(req) additionally requires role admin. Both reload the live user and reject accounts whose status is not active, so deactivation takes effect immediately.
- Roles: admin, central-admin, source, testing. Permission helpers in src/lib (requisitionPermissions, calendarPermissions) are shared by the API and the UI so the UI never offers what the server refuses.
- Source users only ever see requisitions they created; the rule is enforced in the API queries, not just in the UI.

- Report rules in the API: POST /api/reports refuses source users; PATCH and DELETE on /api/reports/[id] allow the testing role only; PATCH also refuses reports older than 10 days.
- Requisition rules in the API: source users may only PATCH their own requisitions and only the intake fields in SOURCE_EDITABLE_FIELDS; DELETE is admin only; closed_at is set when a non-source user sets status Closed.
- Every account is created with must_change_password = true; the layout shows a non-dismissible dialog until it is cleared by a successful password change.
- Destructive actions use the in-app ConfirmModal, never window.confirm, which browsers can silently suppress.
- Deletions, edits and creations are written to audit_logs under the actor's identity.

5. Data model

Table	Purpose and key columns
users	Accounts: name, email, bcrypt hash, role (admin / central-admin / source / testing), status (pending / active / inactive / rejected), must_change_password, reviewed_by / reviewed_at. Shared with sales-portal-next.
test_requisitions	Testing requests: requisition_no (REQ-nnnnnn), model, category, EC/quotation no., offer date, responsible person, source team, date of requisition, target date, rated QTH / power / head / RPM / capacity (+units), status, retest flags, remarks, created_by, submitted_by (name snapshot), closed_at.
pump_test_reports	Report header: report_no (TR-nnnnnn), model, report_format (observation viscosity-chart), test type, rated capacity/head/power/RPM, gearbox/motor info, specific gravity, viscosity, K, NPSHa, witness/inspector/recorder, vibration summary, remarks, tested_by, test_date, requisition_id.
pump_test_report_points	One row per test point: head, RPM, capacity, power, volumetric (VE) and mechanical (ME) efficiency and the other computed columns.
pump_models	Pump model names used as dropdown suggestions.
requisition_attachments	Files attached to a requisition (metadata plus stored file data; downloaded through a dedicated endpoint).
bug_reports	Bug / feature reports from the "Report a Bug" widget: type, severity, title, description, page, status (Open / In Progress / Resolved), optional screenshot, reporter, is_read.
audit_logs	Append-only event trail: user, event_type (login, login_failed, logout, create, update, delete), entity type/id/label, details, IP address, user agent (sign-ins), timestamp.
user_sessions	One row per sign-in: login_at, logout_at, last_seen_at, page_view_count.
page_views	One row per page navigation inside a session.
action_registry	One row per Assign Retest: unmet fields, rated vs measured snapshot, action points, who assigned, who originally raised the test.
calendar_events	Testing Calendar events: title, model, EC/quotation no., responsible person (Sachin Vikash), status (Planned Completed), date, optional start/end time, notes, created_by.

5.1 Things to know before touching the schema

- The users table is shared with sales-portal-next. Treat any change to it with care and coordinate; must_change_password was added here with an additive script.
- Drizzle's .default() and .\$defaultFn() are application-level for most columns (id, created_at, updated_at). Raw SQL inserts must supply id (gen_random_uuid()) and timestamps (now()) explicitly.
- Real foreign keys exist that schema.ts does not model: test_requisitions.created_by -> users.id and users.reviewed_by -> users.id. DELETE /api/users/[userId] clears both inside a transaction before

deleting; copy that pattern for any other deletion path.

- test_requisitions.submitted_by is a name snapshot, not a foreign key, so history stays readable after an account is removed.
- calendar_events.created_by is a foreign key to users with ON DELETE SET NULL.
- Migration convention: additive one-off scripts in scripts/ (ALTER TABLE ... ADD COLUMN IF NOT EXISTS) kept as a permanent record; data-fix and debug scripts are deleted after use.

6. API reference

All routes are under /api and return JSON with snake_case field names. "Signed in" means a valid auth_token cookie. Where a route also applies role rules inside the handler (for example admin-only writes on a route that anyone can read), the purpose column says so. This table is generated from the route files.

Route	Methods	Access	Purpose
/api/access-requests	POST	Public	Public sign-up request; creates a pending user
/api/action-registry	GET	Signed in	List Assign-Retest entries (Admin / Central Admin)
/api/admin-docs/[name]	GET	Admin	Admin-only: sends one documentation PDF (inline, or ?download=1)
/api/audit-log/activity	GET	Admin	Paged event log with filters
/api/audit-log/export	GET	Admin	Audit report as PDF (pdftkit)
/api/audit-log/overview	GET	Admin	Everything the Overview tab draws (adoption, trend, heatmap, grids)
/api/audit-log/sessions	GET	Admin	Sign-in sessions list
/api/audit-log/summary	GET	Admin	Trailing-24h header figures
/api/audit-log/usage	GET	Admin	Per-user usage rollup (sessions, active time, pages)
/api/audit-log/usage/[userId]/pages	GET	Admin	Pages one user visited
/api/audit/pageview	POST	Signed in	Records a page view for the current session
/api/auth/change-password	POST	Signed in	Change own password; clears must_change_password
/api/auth/login	POST	Public	Email + password -> JWT in httpOnly cookie
/api/auth/logout	POST	Signed in	Closes the session, clears the cookie
/api/auth/me	GET	Signed in	Verifies the cookie and returns the live user
/api/bug-reports	GET, POST	Admin	List / file bug and feature reports
/api/bug-reports/[id]	GET, PATCH, DELETE	Admin	Read, change status, delete one report
/api/bug-reports/[id]/screenshot	GET	Admin	Download the attached screenshot
/api/bug-reports/unread-count	GET	Admin	Unread count for the sidebar badge
/api/calendar-events	GET, POST	Signed in	List events (anyone); create (Admin)
/api/calendar-events/[id]	PATCH, DELETE	Signed in	Update / delete an event (Admin)
/api/calendar-events/ec-options	GET	Signed in	EC / quotation numbers for the event form search

/api/calendar-events/requisition-options	GET	Signed in	
/api/health	GET	Public	Liveness check
/api/overview	GET	Signed in	Dashboard figures with date, scope, category, model, status filters
/api/performance	GET	Signed in	VE & ME per model against acceptance criteria; improvement list
/api/pump-models	GET	Signed in	Known pump models (dropdown suggestions)
/api/pumps	GET	Signed in	Report Compilation: pump groups, 50 per page
/api/pumps/[model]	GET	Signed in	Everything for one pump model
/api/reports	GET, POST	Signed in	List / create test reports
/api/reports/[id]	GET, PATCH, DELETE	Signed in	Read / edit / delete one report
/api/reports/[id]/assign-retest	POST	Signed in	Raise a retest requisition + action-registry row
/api/reports/grouped	GET	Signed in	Report Archive grouped by pump
/api/reports/list	GET	Signed in	Flat paged report list (drill-downs)
/api/requisitions	GET, POST	Signed in	List (filtered, 25/page) / create requisitions
/api/requisitions/[id]	GET, PATCH, DELETE	Admin	Read / edit / delete one requisition
/api/requisitions/[id]/attachments	GET, POST	Signed in	List / upload attachments
/api/requisitions/[id]/attachments/[attachmentId]	GET, DELETE	Signed in	Download / delete one attachment
/api/requisitions/dedup-check	GET	Signed in	Check for earlier tests of the same model/EC
/api/requisitions/filter-options	GET	Signed in	Distinct models, submitters, months for filters
/api/requisitions/target-date-alerts	GET	Signed in	Items due within 5 days for the bell
/api/users	GET, POST	Admin	List users (paged) / add user (Admin)
/api/users/[userId]	PATCH, DELETE	Admin	Edit role / activate / deactivate / delete a user (Admin)
/api/users/[userId]/password	PATCH	Admin	Admin sets a user's password

List endpoints are server-paginated through `src/lib/pagination.ts` (25 rows per page unless stated) and return { entries, total, page, page_size }.

7. Core business logic modules (src/lib)

Module	Responsibility
testReportCalc.ts	The live formula engine ported from the Excel workbook (sheets "v notch" and "UPDATE SHEET V-NOTCH (2)"). Only K for Given CPS feeds slip and efficiency; specific gravity and viscosity are informational.

requirementCheck.ts	Met / missed for Head, Capacity (reach at least) and Power (ceiling).
reportCategory.ts	Derives a report's category from its remarks, EC number or linked requisition.
reportEnrichment.ts	Adds category, point counts, max VE / ME and unmet fields to report rows.
veMeAcceptance.ts	Acceptance table per model, impossible-reading thresholds (VE > 150, ME > 100).
modelKey.ts	Normalises model names (H20, h-20, H 20) to one key for every lookup.
unitConversion.ts	Live converted-unit hints for Capacity and Head on the requisition form. Display only; the stored value is never changed.
requisitionAssignment.ts	Default Responsible Person by category (EC Based -> Vikash, otherwise Sachin), applied by the form and enforced by POST /api/requisitions.
adminDocs.ts	The list of documentation PDFs offered on Admin > Documentation.
formUtils.ts	Date formatting and the effective target date (explicit or date + 7 days).
reportEditWindow.ts	10-day edit window.
reportDraft.ts, reportFieldMaps.ts	Cross-form draft carry-over and snake_case / camelCase field maps.
activeTime.ts	Active-time rule: gaps of 15 minutes or less between a user's consecutive actions.
auditRange.ts	IST date windows for every audit query.
raisedBy.ts, requisitionFilters.ts, requisitionLookup.ts	Raiser grouping and requisition filter / lookup helpers.
auth.ts, audit.ts	JWT handling and audit-log writing.

7.1 Active-time algorithm

Implemented as one SQL derived table in src/lib/activeTime.ts. For each user, audit_logs rows (excluding login_failed) are ordered by time; each row adds the gap to the previous row when that gap is at most 15 minutes (a window function: lag over partition by user_id). A gap is credited to the later action's IST day and to the range that contains it. The same derived table feeds Active hours, the Top 3 podium, the per-user day grids, Most active user and the Usage by User tab, so all figures agree.

7.2 Dashboard filters

GET /api/overview accepts from, to, mine, category, model and status. They are folded into two shared SQL conditions (one for requisitions, one for reports) so every figure, trend and list respects them. A report's category is derived rather than stored, so matching report ids are computed once per request. Reports have no status: status=Closed keeps reports, any other status excludes them. Model matching uses the normalised key.

8. Frontend

- Pages are client components that load data through services and hold state with React hooks. Forms use react-hook-form with zod schemas.
- Infinite scroll (Testing Summary): an IntersectionObserver on a sentinel row appends the next 25 rows; guards stop duplicate loads.
- Drill-down modal (DrillDownModal): given a /dashboard?... or /reports?... link it fetches the matching list and renders it in a pop-up. The Overview intercepts clicks on those links.

- Charts are hand-built SVG components (LineChart, StackedBarChart, DonutChart, Heatmap, BarList) with a table twin for accessibility.
- Theming: design tokens as CSS variables with light and dark palettes; Tailwind classes map to the tokens. The `tw-reset` class scopes a clean button reset to a screen. Print styles force the light palette.
- Do not rely on a `useState(localStorage...)` initialiser for state that must reflect the live session; pair it with a mount effect (see `DashboardLayout`).
- Test report PDFs use `window.print()`. The layout's fixed height and hidden overflow must be reset inside the print media query or exports truncate after one screen.

9. Audit and telemetry

- `audit_logs` receives login, login_failed, logout and every create / update / delete made through the API (`logAudit`), with IP address and, for sign-ins, the user agent.
- `user_sessions` is opened at sign-in and closed at sign-out; `/api/audit/pageview` (called on every route change) updates `last_seen_at` and inserts a `page_views` row.
- The Audit Log overview aggregates in one request (GET `/api/audit-log/overview`): adoption (accounts, active, never signed in, dormant, sessions, page views), trend, heat map, per-user grids, devices, browsers and IPs. Day and hour buckets are in IST.
- The audit PDF is generated server-side with `pdfkit`. Version 0.20 crashes on Vercel, which is why `pdfkit` is pinned to 0.15.2.

10. Deployment and operations

- Branch main deploys to Vercel automatically on push.
- Database changes: write an additive `scripts/add-*.mjs` script, run it with `dotenv` against the database, commit the script, then deploy the code that uses the new column.
- Both portals read the same database. Take care with changes that touch users.
- A Vercel "Security Checkpoint" page returned to rapid automated requests is bot protection, not an application fault; it clears after a pause.
- Local verification: run the dev server, sign in, and type-check with `npx tsc --noEmit`. There is no automated test framework.

11. Conventions and known risks

- Type-check before every commit; commit and push to main (the live site deploys from it).
- Throwaway scripts and test database rows must be deleted straight after use.
- No automated tests exist; regressions are caught only by manual checking. Adding tests around `testReportCalc`, `requirementCheck` and the audit SQL would be the highest-value first step.
- The shared users table and shared JWT secret couple the two portals; a secret rotation or schema change must be coordinated.
- Active time is conservative because the audit log records sign-ins and writes only. Recording page views as actions, or adding a heartbeat, would raise accuracy.
- Role rules are duplicated by design in API and UI helpers; always change both through the shared helper.